Lecture 1 · Foundations / SDLC / Git

SDLC: พัฒนาระบบ ไม่ใช่แค่เขียนโค้ด

Software engineering: พัฒนาระบบให้แก้ปัญหาผู้ใช้ ทดสอบได้ ปลอดภัยและดูแลต่อได้; โค้ดรันได้ ≠ ผู้ใช้ยอมรับ Planning → Requirements → Specification → Design → Implementation → Testing → Deployment → Maintenance **Planning**: เหตุผล ขอบเขตและความเสี่ยง; **Req**: ระบบต้องทำอะไร; **Spec**: ตกลงพฤติกรรม; **Design**: ทำอย่างไร;

Test: พิสูจน์ผล; **Deploy**: ตั้งค่า นำขึ้นใช้ เฝ้าดู/ย้อนรุ่น;

Maintain: แก้และปรับระบบ **เพิ่มเติม:** Feasibility: ทำได้ทางเทคนิค คุ้มทุน และมีเวลา/ทรัพยากรพอ

Process models / Agile values

Model	จุดเด่น / ข้อควรระวัง
Waterfall	แผนและ sign-off ชัด; เหมาะกับข้อกำหนดคงที่ แต่ feedback ช้า เปลี่ยนแพง
Incremental	ส่ง vertical slice ที่ใช้ได้ทีละส่วน; คุม architecture และ integration
Iterative	ปรับ solution เดิมซ้ำตาม feedback
Scrum	sprint goal, timebox, increment; ตรวจสอบและปรับตัว
Kanban	งานไหลต่อเนื่อง; จำกัด WIP วัด cycle time/throughput

Agile ให้ความสำคัญมากกว่า: คนและปฏิสัมพันธ์ > กระบวนการ/เครื่องมือ; ซอฟต์แวร์ที่ใช้ได้ > เอกสารครบถ้วน; ร่วมมือกับลูกค้า > ต่อรองสัญญา; ตอบรับการเปลี่ยนแปลง > ยึดแผน ด้านขวายังมีคุณค่า **Vertical slice**: UI + API + DB + test ใช้ได้ครบเส้นทาง; ไม่ใช่ทำ DB อย่างเดียวแล้วค่อยต่อ UI **Definition of Done (DoD)**: เกณฑ์เสร็จจริง เช่น ผ่าน AC/tests/review แก้ข้อบกพร่องสำคัญ เอกสารตรง และรวมงานตามกติกา

AI engineering / SDD + TDD

Agent: observe → plan → tool/edit/test → inspect → adjust; ต่างจาก chatbot ที่เน้นตอบข้อความ **Prompt ที่ชัด**: ให้บริบท ข้อกำหนด/contract ข้อจำกัด ขอบเขตไฟล์ วิธีทดสอบและหลักฐาน; แบ่งงานก่อนทำ แล้วตรวจ diff กับผลจริง คนรับผิดชอบเป้าหมาย การยอมรับและความเสี่ยง; คำว่า “ผ่าน” จาก agent ไม่ใช่หลักฐาน ระวัง hallucination, regression, secrets, dependencies และ docs ที่ล้าสมัย

Architecture / Git พื้นฐาน

React UI → HTTP API/Express → service/กฎธุรกิจ → Prisma ORM → PostgreSQL; UI ไม่ต่อฐานข้อมูลโดยตรง **Working tree** แก้ไฟล์ → **add** เลือก snapshot เข้า **index** → **commit** บันทึกใน local → **push** ส่งไป remote Issue → branch → commit → PR → review → staging → main; feature แยกงาน; staging รวม/ทดสอบ; main คืองานที่ยอมรับ; README บอก setup/run/test/env/กติกา

git log --oneline --graph --decorate --all

log flags: **--oneline** SHA สั้น+หัวเรื่อง; **--graph** เส้น ancestry; **--decorate** ชื่อ refs; **--all** ทุก refs ของเครื่อง ไม่ fetch ไม่ commit .env/secrets, node_modules, temp; ใช้ .env.example

Lecture 2 · Requirement Engineering

ระดับข้อกำหนด: Business = เป้าหมาย; User/URS = ผู้ใช้ต้องการอะไร/ทำไม; System/SysRS = รวมฮาร์ดแวร์ ส่วนเชื่อมต่อ/กฎระเบียบ; Software/SRS = พฤติกรรมและข้อจำกัดซอฟต์แวร์

ชนิด	คำถาม / ตัวอย่าง
FR	Functional requirement: ระบบทำอะไร
NFR	Non-functional requirement: คุณภาพระดับใด ภายใต้เงื่อนไขใด
BR/ domain	Business/domain rule: กฎธุรกิจ เช่น ไม่เกิน 3 ครั้ง/ภาค
Constraint	ข้อจำกัดทางเลือก เช่น เทคโนโลยีหรือที่ตั้ง server

FR/NFR/BR เป็นชนิด ไม่ใช่ระดับ user/system; กฎหนึ่งผูกกับหลาย FR ได้ ไม่ต้องแยก BR ให้ทุกข้อ **NFR**: performance, security, usability, reliability, maintainability; ระบุตัวชี้วัดเงื่อนไขและเกณฑ์ผ่าน

RE process / Quality checklist

Elicit/analyse → specify → validate → manage changes; หาผู้เกี่ยวข้อง/เป้าหมาย → สอบถาม สังเกต อ่านเอกสาร → บันทึก → ยืนยัน → จัดลำดับ **Fact**: ระบุแล้ว; **Assumption**: ยังไม่ยืนยัน; **Question**: ต้องถาม; แยกสิ่งที่ทำ/ไม่ทำ/ไว้ภายหลัง **Quality**: ชัดเจน ไม่กำกวม มีเรื่องเดียว จำเป็น ครบ สอดคล้อง ทำได้จริง ทดสอบและย้อนรอยได้; ระบุ actor/action/object/condition **Validation**: ทบทวน สร้างต้นแบบหรือออกแบบ tests เพื่อตรวจความเข้าใจ; **Change control**: บันทึกเหตุผล เจ้าของ ผลกระทบ ข้อสรุปและรุ่น

Proposal 8 ช่อง / Evidence chain

Problem · Users · Solution · Expected value · Scope · Risk · Effort · Evidence of completion แยกสิ่งที่มี/ขาดพร้อมหลักฐาน; ข้อเสนอเพิ่มต้องชัดป้าย ไม่เขียนเป็นข้อเท็จจริง Req ID/ต้นทาง → feature/design → Issue/code → AC/test → PR/release → demo; ย้อนถึงไฟล์/ผลจริงได้ **Acceptance criteria (AC)**: เกณฑ์ยอมรับเฉพาะงาน; **DoD**: เกณฑ์เสร็จร่วม; **Definition of Ready (DoR)**: พร้อมเริ่ม ≠ เสร็จ; MoSCoW: Must / Should / Could / Won't (รอบนี้)

วิเคราะห์ข้อกำหนดพร้อมเหตุผล

ปัญหา	ต้องซื้อะไร / แก้อย่างไร
กำกวม	“quick/easy” ไม่มีเกณฑ์ → ตกลง metric/load/threshold
ไม่ atomic	หลาย actor/หลายเป้าหมาย → แยกข้อ แต่ละ trace กลับต้นฉบับ
ไม่ครบ	ขาด precondition/output/error/สิทธิ์ → ถามแต่ละสถานะ
ขัดแย้ง	เช่น limit 3 กับ 4 → ให้ผู้มีอำนาจตัดสิน ไม่เลือกเอง
ตรวจไม่ได้	แปลงคำกว้างเป็นผลลัพธ์ที่วัดได้ พร้อมวิธีวัด
ทำไม่ได้	ตรวจเทคนิค/เวลา/งบ; เจรจาขอบเขต ไม่รับปากลอย ๆ
ย้อนกลับไม่ได้	ใส่ ID/source/owner แล้วโยง AC/design/test

Elicitation: ใครทำ? เมื่อไร? ข้อมูลเข้า/ออก? กฎ/สิทธิ์? ข้อยกเว้น? นอกขอบเขต? ใช้ interview/observation/docs/prototype ตรวจความเข้าใจร่วม **BR** → **FR**: จำกัด 3 รายการ/ภาค → ระบบปฏิเสธรายการที่ 4; NFR ระบุระดับคุณภาพ **เพิ่มเติม:** AC: Given มี 3 รายการในภาคนี้; When ขอเพิ่ม; Then ปฏิเสธ/ยังเป็น 3; ถามว่าการยกเลิกนับไหม

Lecture 3 · Specifications / SDD / Tests

SRS → SDS → STS

SRS: ต้องการอะไร; **SDS**: ออกแบบอย่างไร; **STS**: ตรวจอะไร ด้วยข้อมูลใด คาดผลอย่างไร และเก็บหลักฐานใด SRS → feature inventory → Req-to-feature map → system SDS → vertical slice → feature contract → code → test/review → integrate → main **System SDS**: architecture/stack/data, auth, มาตรฐาน API/UI/error และ NFR ร่วม **Feature SDS**: scope, Req, workflow/state/BR, data/API/UI, สิทธิ์/validation/error, AC/tests **Contract 7**: inputs → validations → behavior → persistence → side effects → errors → outputs Invoice: รับ customer/date/lines → ตรวจ customer, qty > 0, price ≥ 0 → คำนวณ → บันทึกหัว/รายการใน transaction เดียว; ผิด → ไม่บันทึก **เพิ่มเติม:** receipt เป็น side effect ที่ต้องตกลง

Tests / Red-Green-Refactor

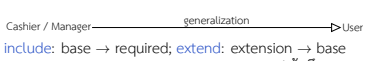
RED: test ล้มเหลวด้วยเหตุผลที่ตั้งใจ → GREEN: เขียนโค้ดเท่าที่จำเป็นให้ผ่าน → REFACTOR: ปรับโครงสร้างโดย tests ยังผ่าน **AAA**: Arrange เตรียมข้อมูล → Act เรียกใช้ → Assert ตรวจผล; ครอบคลุมปกติ/ผิดเงื่อนไข/ขอบเขต/error/สิทธิ์; ต้องจับ defect ได้ ไม่เขียนตาม implementation อย่างเดียว **STS row**: test ID + Req/AC → precondition/data → steps/input → expected UI/API/DB → file/result **เพิ่มเติม:** ค่าขอบเขต (boundary): คะแนน 0-100 → ทดสอบ −1/0/100/101; 0 และ 100 ผ่าน ส่วน −1 และ 101 ถูกปฏิเสธ

ระดับ/ชนิด	ตรวจอะไร
Unit	function/class/component; อาจแยก deps ด้วย mock
Integration	modules/API/DB/interfaces ทำงานร่วมกันจริง
System	ระบบรวมครบ SRS ทั้ง FR/NFR
E2E	เส้นทางผู้ใช้ผ่าน UI → API → DB/external
UAT	ผู้ใช้/ธุรกิจยอมรับตาม need/AC และ sign-off
Regression	ทดสอบของเดิมหลังแก้ไข; ใช้ได้ทุกระดับ
Smoke	เปิดระบบและใช้งานหลักได้หลัง setup/release
Special	load/performance, security, compatibility, recovery

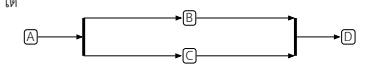
Verification: ทำตรง spec ไหม; **Validation**: ตรงความต้องการผู้ใช้ไหม; coverage สูง ≠ tests จับข้อผิดพลาดได้ดี **Sync**: รอผลตอบ; **Async**: ส่งงานแล้วรับผลภายหลัง; ระบุการลองซ้ำ (retry) และ idempotency: ส่งซ้ำแล้วไม่เกิดผลซ้ำเกินที่ตกลง **POS recovery/outbox**: เก็บ payment attempt; ผลไม่ชัดตรวจ PSP ก่อนเก็บเงินซ้ำ; บันทึก sale + pending inventory แบบ atomic; ส่ง async รอ ack/retry ใช้ transaction ID กัน stock ซ้ำ **BDD**: เริ่มจากพฤติกรรมด้วย Gherkin (outside-in); **TDD**: ให้ tests นำการออกแบบ (inside-out); ใช้ร่วมกันได้

Lecture 4 · UML / Feature Design / UI

Use Case: actor มี goal ใน boundary; association เป็นเส้นทึบไม่มีหัวลูกศร

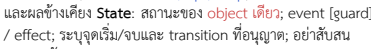


include: base → required; extend: extension → base [condition]; generalization: child → parent (เส้นทึบ; สามเหลี่ยมไปร่งชี้ parent) **Activity**: workflow ครบทางเลือก; • เริ่ม, action มุมมน, ◇ decision/merge + [guard], bar = fork/join ขนาน, ⊙ จบ; swimlane แบ่งผู้รับผิดชอบ; ลูกศรข้าม lane ได้



fork: ทำ B/C ขนาน; join: รอครบทั้งคู่

Sequence: scenario หนึ่งใน workflow; participant/lifeline อยู่ด้านบน เวลาลงล่าง; call เส้นทึบ/return เส้นประ; activation คือช่วงทำงาน; alt [condition], opt, loop; แสดงระบบภายนอกและผลข้างเคียง **State**: สถานะของ object เดียว; event [guard] / effect; ระบุจุดเริ่ม/จบและ transition ที่อนุญาต; อย่าสับสน action กับ state



Class Diagram: กล่องคลาสมีชื่อและ attributes:type; ไม่แสดง methods. แสดง association พร้อม multiplicity, inheritance (child→parent), composition (◆ ที่ whole), aggregation (◇) และ external calls; จำกัดความสัมพันธ์ที่ Header/Line Item ไว้เฉพาะชั้นแรก **Scenario**: basic = ปกติ; alternative = ทางเลือกที่ถูกต้อง; exception = ผิดพลาด; Activity ≠ Sequence; Use Case ≠ flowchart

Generic UI: List / Form / LoV

List: title/role, search/filter/sort, columns, select/open/create, pagination, empty/loading/error; **Form**: header/lines/total/status/actions **สัญลักษณ์สอบ**: * นำหน้าช่องบังคับ; textbox แก้ได้; ชัดเจน = read-only/calculated; เลือก FK ด้วย LoV; รายการย่อยมี add/remove และ validation **ทำ UI**: Req → header/lines → label+textbox → */edit/RO/LoV → controls ตาม role/state และ errors **ตามสถานะ**: KU Draft แก้/Save/Submit; Done ล็อก; รอ/สำเร็จ/ผิดพลาดตามcontract; state ยึดแต่ละระบบ **Design system**: สี/ฟอนต์/ระยะ/components/grid/breakpoints, contrast/keyboard/focus; ตารางยาวเลื่อนแนวนอนได้ ไม่เติม split payment/cancel/refund หากโจทย์ไม่ให้; ทุก field/button/transition ต้องโยง Req/สิทธิ์ได้

จุดเน้นเมื่อวาดระบบตัวอย่าง

ระบบ	จุดที่ควรแสดงในแบบจำลอง
POS	ทางเลือกเมื่อชำระไม่ผ่าน; การยกเลิก/คืนเงินและผลต่อ stock
Hotel	actor และขอบเขต; payment, Map, Email และสิทธิ์ดูแลโรงแรมตน
Ticket	role/guard ของ state transition; action history เพิ่มได้อย่างเดียว
Grades	งานที่ต้องเกิดหลัง submit; ใช้ generalization เมื่อมี parent-child ที่ระบุ

ตรวจรูป: ตรงข้อกำหนด; guard/ผลข้างเคียงครบ; กรณีผิดกลับแก้ได้; ไม่แต่ง state/สิทธิ์

Lecture 5 · Security Engineering

Security landscape / CIA

Actor: ผู้กระทำ; **Asset**: สิ่งที่ต้องปกป้อง เช่น บัญชี/รายงาน; **Attack surface**: จุดรับการโจมตี เช่น API/upload; **Trust boundary**: ขอบเขตความเชื่อถือ เช่น browser → server C: Confidentiality ป้องกันการเห็นข้อมูลโดยไม่มีสิทธิ์; I: Integrity ป้องกันการแก้ไขโดยมิชอบ; A: Availability ผู้มีสิทธิ์ยังใช้ระบบได้ **Secure by design**: least privilege, defense in depth, fail safe/default deny, ออกแบบเรียบง่าย แบ่งส่วน ลดจุดโจมตี; ค่าเริ่มต้นปลอดภัยและยังใช้งานได้

Authentication ≠ Authorization

Authentication (AuthN): ยืนยันว่าเป็นใคร; **Authorization (AuthZ)**: ตรวจว่าทำได้ไหม ตามผู้ใช้ การกระทำ ทรัพยากรและ state; ตรวจที่ backend ทุก request **RBAC**: สิทธิ์ตาม role; **ABAC**: ตามคุณลักษณะผู้ใช้/ทรัพยากร/บริบท; **ACL**: รายการสิทธิ์ต่อทรัพยากร; ตรวจ ownership เทียบตัวตนที่ server ยืนยัน ไม่เชื่อคำอ้างว่าเป็นเจ้าของจาก client **Password**: slow hash + salt เฉพาะแต่ละรหัส; ไม่เก็บ/บันทึก plaintext. Base64 แค่แปลงรูปแบบ ไม่ใช่ encryption; encryption ถอดคืนด้วย key; hash ทางเดียว ใช้ตรวจเทียบ **Basic**: ส่ง credentials แบบ Base64 ต้องใช้ HTTPS; **Digest**: ตอบด้วย hash/nonce เป็นวิธีเก่า ไม่แทน HTTPS **Session**: server เก็บสถานะ ส่ง session ID ผ่าน cookie; หมดอายุ/logout แล้วใช้ต่อไม่ได้; current-user คืน profile ไม่คืน secret **JWT**: token ที่ลงลายเซ็น ไม่ได้เข้ารหัสเนื้อหาเสมอ; ตรวจลายเซ็น/วันหมดอายุ; ต้องออกแบบเพิกถอนเอง; access อายุสั้น ส่วน refresh ต้องปกป้องและเพิกถอนได้ **API key**: secret ของแอป/บริการ; **OAuth2**: มอบสิทธิ์ให้แอปอื่น; **OIDC**: เพิ่มการยืนยันตัวตนบน OAuth2; **SSO**: sign-in ครั้งเดียวใช้หลายระบบ

OWASP / Vulnerability / Control

ช่องโหว่	การป้องกัน / การทดสอบเชิงลบ
Broken access	ตรวจ role/owner/state ที่ backend; เรียก API ข้ามคน/role → ปฏิเสธ ไม่รั่วข้อมูล
Auth failure	error กลาง, rate limit, salted hash, expiry; ลอง wrong/inactive/logout แล้วเข้าตรง
SQLi	parameterized query/ORM; allowlist ชื่อ field; ' OR '1'='1 ต้องเป็นข้อมูล ไม่ใช่ SQL
XSS	encode output, sanitise rich text; ไม่ใช้ HTML ที่ไม่ปลอดภัย; payload ต้องไม่รัน
CSRF	browser ส่ง cookie ข้าม site; ใช้ token/SameSite/Origin check; ปฏิเสธ forged write
SSRF	server ดึง URL จาก input; allowlist ปลายทาง/บล็อก internal/metadata/loopback
Upload	server ตรวจชนิด/ขนาด/เนื้อหา; เก็บแบบรันไม่ได้/ดาวน์โหลดปลอดภัย; script ปลอม PDF ต้องไม่รัน
Config/ secrets	HTTPS, secure cookie, CORS เคร่งครัด; ไม่เผย debug/.env; เปลี่ยน secret ที่รั่ว
Deps	patch/audit packages; จำกัดสิทธิ์ DB/network; ปกป้อง backups

ซ่อนปุ่มไม่ใช่การควบคุมสิทธิ์; frontend validation ช่วย UX; backend validation คุม trust boundary **Status**: 400 ข้อมูลผิด; 401 ยังไม่ยืนยันตัวตน; 403 ไม่มีสิทธิ์; 404 ไม่พบ/ปิดบังตามกติกา; 409 ขัดแย้ง; 500 ข้อผิดพลาดกลาง; log correlation ID ไม่ใช่ password/token **เพิ่มเติม:** เมื่อปฏิเสธ ให้ตรวจ error/status และข้อมูลธุรกิจไม่เปลี่ยน; audit log ยังเกิดได้ตามcontract

ตัวอย่างตรวจสิทธิ์ผ่าน API โดยตรง

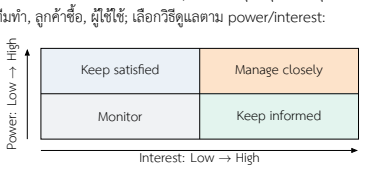
ทดสอบ	ผลที่ต้องตรวจตาม policy/contract
AuthN/gate	valid ผ่าน; wrong/inactive/ไม่ login ถูกปฏิเสธ; password gate ต้องคุม API ตรง
ผิด role	เรียก endpoint ตรงไม่ได้; ไม่มี business write
ข้าม owner	A เปลี่ยน ID ไปขอข้อมูล B → ไม่เผยข้อมูล/จำนวน/ไฟล์
logout/expiry	ใช้ session เดิมซ้ำไม่ได้ แม้เปิด URL/API ตรง
input/duplicate	error ปลอดภัย; DB ไม่ค้างครึ่งชุด; ข้อมูลเดิมไม่เสีย
log/config	ไม่พบ token/password/stack trace ใน response หรือ log ที่เปิดเผย

แนวตรวจ: AuthN → AuthZ → validation → ลงมือ/บันทึก; ไม่เชื่อ role จาก client; ลำดับย่อยขึ้นกับข้อมูลที่ต้องใช้ตรวจสิทธิ์ **Defense in depth**: ดัดไฟล์หลุด validation ต้องยังรับไม่ได้ และอ่านข้าม owner ไม่ได้

Lecture 6 · Project Initiation

Initiate → Plan → Execute → Close; เป็นวงจรโครงการ ส่วน SDLC อยู่ภายในโครงการ **Initiation 6**: goals, scope, deliverables, success criteria, stakeholders, resources → charter → อนุมัติ → วางแผน **Goal**: ผลที่ต้องการ;

Deliverable: สิ่งส่งมอบ; **Success criteria**: หลักฐานว่าสำเร็จ; launch = ส่งมอบ, landing = วัดผล **SMART**: Specific ชัด, Measurable วัดได้, Attainable ทำได้, Relevant สอดคล้อง, Time-bound มีเส้นตาย; ลดมัยธฐานเวลาตอบครั้งแรก 20% ภายในภาคนี้ด้วย routing อัตโนมัติ **OKR**: Objective = ทิศทาง; KRs = ผลที่วัดได้; ท้าทายและเชื่อมองค์กร → แผนก → โครงการ; KR ไม่ใช่รายการงาน ROI=(Gain-Cost)/Cost ×100%; Gain = ผลตอบแทน, Cost = ต้นทุนทั้งหมดในช่วงเดียวกัน; พิจารณาคุณค่าที่ตีราคาไม่ได้แยก G=10,000; C=6,000+25×12=6,300 → ROI=58.7% **Scope**: ตกลงสิ่งที่ทำ/ไม่ทำและข้อจำกัด; **Scope creep**: ขอบเขตเพิ่มโดยไม่ควบคุม ทั้งจากลูกค้าหรือทีม; ขอเปลี่ยน → วิเคราะห์ผล → เสนอทางเลือก → อนุมัติ → บันทึก **Triangle**: scope/time/cost; เมื่อตรึงด้านหนึ่งต้องปรับด้านอื่น ไม่รับงานเพิ่มโดยไม่ประเมิน **Stakeholders**: sponsor อนุมัติทุน, PM คุมแผน, ทีมทำ, ลูกค้าซื้อ, ผู้ใช้ใช้; เลือกวิธีดูแลตาม power/interest:



Charter: เหตุผล/เป้าหมาย/ขอบเขต/สิ่งส่งมอบ/เกณฑ์สำเร็จ/ทีม/ทรัพยากร/ทุน/กำหนดการ/สมมติฐาน/ความเสี่ยง/ผู้อนุมัติ

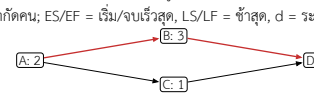
เพิ่มเติม: เว็บที่ใช่ได้ = deliverable; ลดเวลารอ = goal; success criteria ระบุค่าตั้งต้น เป้าหมายและเส้นตายที่ตกลง

Lecture 6 · Project Planning

WBS / Schedule / Responsibility

WBS: project → milestone/deliverable → task → subtask; แยกแบบ top-down/bottom-up. **Milestone**: จุดสำเร็จ duration = 0; task มี owner และ duration **Gantt**: WBS no./task/owner/start/due + แถบเวลา; ทำขนานได้เมื่อ dependency และทรัพยากรพร้อม; roadmap เน้น goal/milestone ภาพรวม **Critical path (CP)**: เส้นทางงานที่พึ่งกันและใช้เวลารวมยาวสุด; **float**: เวลาที่เลื่อนได้โดยไม่เลื่อนวันจบ;

CP มี float = 0 เมื่อกำหนดจบเท่ากับวันจบเร็วสุด เริ่มเวลา 0: EF=ES+d; ES=max(EF ของ pred); LS=LF-d; LF=min(LS ของ succ); Float=LS-ES **เพิ่มเติม:** สูตร/กราฟ: งานต่อกัน ไม่มี lag/ข้อจำกัดคน; ES/EF = เริ่ม/จบเร็วสุด, LS/LF = ช้าสุด, d = ระยะเวลา



A → B → D = 7 วัน (CP)

A → C → D = 5 วัน; C float = 2 วัน

Task	ES	EF	LS	LF	Float
A	0	2	0	2	0
B	2	5	2	5	0
C	2	3	4	5	2
D	5	7	5	7	0

วันที่แบบ inclusive: นับวันเริ่มเป็นวันที่ 1; Due = วันทำงานลำดับที่ d; งานถัดไปเริ่มวันทำงานถัดไป ไม่นับวันหยุด/ใช้คนข้อนเกิน กำลัง **Effort**: person-hours/days ≠ **duration**: เวลาที่งานใช้; Brooks: เพิ่มคนในงานที่ล่าช้าอาจยิ่งช้า PERT $E=(O+4M+P)/6$; $SD=(P-O)/6$; O ดีสุด, M น่าจะเป็น, P แย่สุด; E = ค่าคาดหมาย, SD = ส่วนเบี่ยงเบนมาตรฐาน;

เพิ่มเติม: เมื่อประมาณการอิสระ $SD_{total}=\sqrt{\sum_i SD_i^2}$; ใช้หน่วยเดียวกัน **Buffer**: task buffer เผื่อ delay ที่คุมไม่ได้ เช่น รออนุมัติภายนอก; project buffer เผื่อรวมท้ายโครงการ; ไม่ใส่ทุก task ซ้ำซ้อน **RACI**: R ลงมือ, A รับผิดชอบผล, C ปรึกษาก่อน, I แจ้ง; ให้ A ชัดคนเดียว/งานและมี R; ไม่เกี่ยวใช่ “–”; ระวัง R กระจุกตัว **Risk**: ระบุ → วิเคราะห์โอกาส/ผลกระทบ → จัดลำดับ → ตั้งเจ้าของ → รับมือ → ติดตาม; **เพิ่มเติม:** risk อาจเกิด ส่วน issue เกิดแล้ว **Kick-off**: เป้าหมาย/ขอบเขต/บทบาท/tools/process/สิ่งต้องทำ; charter = ทำไม, plan = อย่างไร/เมื่อไร, risk register = อะไรอาจผิด

Lecture 7 · Execution / Agile / Scrum

MVP: ชิ้นเล็กสุดที่ทดสอบคุณค่าได้ → build-measure-learn → pivot/persevere; ไม่ใช่ phase 1 ที่ยังใช้ไม่ได้ **Pair**: driver เขียน/navigator ตรวจ สลับกัน; **DevOps**: dev + ops ร่วมดูแล delivery ใช้ automation/feedback; Water-Scrum-Fall ยังติด batch/approval ที่หัวท้าย **PO**: คุณค่า/วิสัยทัศน์/เรียง backlog/ตัดสินใจเรื่องข้อกำหนด/ยอมรับ/ส่งมอบ; **SM**: อำนวยความสะดวก คุมเวลา แก้อุปสรรค ไม่สั่งแบ่งงาน/ตัดสิน product แทน PO; **Developers**: จัดการตนเอง ข้ามทักษะ เลือกวิธีทำ/คน/capacity **Artifacts 3**: product backlog, sprint backlog, Done increment; **Events 5**: Sprint, Planning, Daily, Review, Retrospective

Story / Backlog refinement

As a <role>, I need <function>, so that <value>; role ไม่ใช่ชื่อคน; มี description/details/assumptions/AC Given <precondition> When <event> Then <observable outcome>; And/But เสริม; ทดสอบบวก/ลบ ไม่กำหนด implementation เกินความต้องการ **INVEST**: Independent, Negotiable, Valuable, Estimable, Small, Testable; **Epic**: ใหญ่เกิน sprint ต้องแยกก่อน **Groom/refine**: ปฏิเสธ/พักใน icebox/จัด backlog; PO เรียงคุณค่า ทีมแยกและชี้แจงงานล่วงหน้าราว 2 sprints; DoR = พร้อมเริ่ม **Metadata**: Assignees, Labels (ชนิด), Milestone (sprint), Estimate (points); enhancement เพิ่มพฤติกรรม, bug แก้ผิด, technical debt ปรับโครง/refactor/env/dependencies

Sprint planning / execution

Points: effort/complexity/uncertainty แบบสัมพัทธ์; Fibonacci 1,2,3,5,8,13,21; anchor 3: +1 ขึ้น = 5, +2 = 8, −1 = 2; ไม่ใช่ชั่วโมง/ไม่เทียบข้ามทีม **Velocity**: points ที่ทีมทำเสร็จต่อ sprint; ใช้ประวัติร่วมกับกำลังทีม (capacity); เลือกตาม goal, priority, dependencies และ $\sum$points ≤ capacity; ไม่เติมงานเพียงเพราะยังมีที่ **Planning**: PO ให้ goal/ลำดับ; ทีมประเมินและเลือกงาน/วิธีทำ; SM คุมเวลา; sprint backlog = forecast + plan; มักยาว 2 สัปดาห์ **Board**: New → Icebox/Backlog → Sprint Backlog → In Progress → Review/QA → Done; WIP 1/คน เว้นติดขัด; เลือกงานสำคัญสุด; เสร็จ → PR → merge/Done ตามกติกา **Daily**: timebox 15 นาที; คุยความคืบหน้า เป้าหมายวันนี้และอุปสรรค; วางแผนร่วมกัน ไม่ใช่รายงานหัวหน้า **Review**: demo/AC กับ PO/ผู้เกี่ยวข้อง; feedback → Issue; **Retro**: ทีม/SM ทบทวนดี/ไม่ดี/สิ่งที่จะเปลี่ยน แล้วนำไปปรับ sprint หน้า **Burndown**: x = เวลา, y = points คงเหลือ; ลดเมื่อ Done ไม่ใช่งานทำ; metrics ต้องมี baseline และนำไปปรับงานได้ **จบ sprint**: งานไม่เริ่มคืน backlog; งานไม่ผ่าน DoD ไม่ถือว่า Done ให้ประเมินและจัดลำดับใหม่; บันทึกส่วนที่เสร็จกับงานค้างแยกกัน **Docs**: ข้อกำหนด/design/technical/QA/user/SOP; **Org**: traditional หัวหน้าเดียว, matrix หลายสาย; Conway: โครงทีมสะท้อนระบบ

Metrics: ความหมาย ไม่ใช่แค่ชื่อ

Metric	ใช้ตัดสินใจอะไร
Mean Lead Time	เวลา idea → production; ยาว → หาเวลารอ/คอขวด
Release frequency	ส่งขึ้นใช้งานบ่อยแค่ไหน; batch ใหญ่ทำให้ feedback ช้า
Change failure rate	สัดส่วน release/change ที่ก่อปัญหา; ไม่ใช่จำนวน tests ที่ผ่าน
MTTR	เวลาเฉลี่ยฟื้นระบบหลัง failure; ดูทั้งตรวจพบและกู้คืน

เพิ่มเติม: ระบุจุดวัด หน่วย ช่วงเวลาและ baseline; จำนวน commits/บรรทัดโค้ด ≠ คุณค่า

วิศิษฐ์ สุวรรณเนาว์ 67070501042

Practice Midterm · Part 1

Part 1 รวม 25 คะแนน: วิเคราะห์ข้อกำหนด ข้อเสนอโครงการ RBAC และความปลอดภัย

1) วิเคราะห์ข้อกำหนดให้ครบ 5 ขั้น

จำแนกชนิด → อธิบายจุดประสงค์ → ประเมินคุณภาพพร้อมเหตุผล → วิธีตรวจ/ผลที่คาด → เสนอแก้; แยกข้อเท็จจริงจากข้อเสนอ

ข้อกำหนด	แนววิเคราะห์
Browse open	FR; นักศึกษาที่ login แล้วดู workshop ที่เปิดรับ; เครียม open/closed → เห็นเฉพาะ open
“Quick”	NFR; กำกวมเพราะไม่มีตัวชี้วัด/ภาระงาน ต้องตกลงก่อนทดสอบ
≤3/semester	BR/domain; ทดสอบมี 2 เพิ่มเป็น 3 ได้; มี 3 แล้วเพิ่มครั้งที่ 4 ต้องปฏิเสธ
Cancel + list	รวมสอง FR; แยกนักศึกษายกเลิกกับผู้ประสานงานดูรายชื่อ; ถามขอบเขตสิทธิ์ให้ชัด

เพิ่มเติม: ตัวอย่าง NFR ที่วัดได้: เมื่อมีผู้ใช้พร้อมกัน 100 คน คำขอสมัครที่ถูกต้องอย่างน้อย 95% ได้รับการยืนยันภายใน 2 s; ต้องตกลงจุดวัด เครื่องข่าย ชุดข้อมูลและกรณีผิดพลาดก่อนใช้ **FR template**: The system shall allow <actor> to <action/object> when <condition>. ระบุผลและข้อยกเว้นที่ตรวจได้

“quick/easy/reliable” ยังวัดไม่ได้; ตกลงตัวชี้วัดก่อนระบุตัวเลข; test ต้องมี input/state, action และ expected result ไม่ใช่แค่ “ทำ unit test”

2) WorkshopHub proposal: 8 ช่อง

Element	หลักฐาน / สิ่งที่ขาด
Problem	หลายฟอร์ม รวบรวมด้วยมือ → ข้อมูลซ้ำ/ล่าช้า
Users	ระบุ Student/Coordinator; ต้องถามผู้อนุมัติ
Solution	เว็บดู workshop/สมัคร/จัดการผู้เข้าร่วม
Value	ลดข้อผิดพลาด/งานธุรการ; ขาด metric/baseline
Scope	รุ่นแรกสมัครฟรี; payment/certificates ไว้ภายหลัง
Risk	ไม่ระบุ; เสนอเรื่องซ้ำซ้อน/สมัครพร้อมกัน/privacy
Effort	“ก่อนภาคหน้า” เป็น deadline ไม่ใช่ effort estimate
Evidence	ยังไม่มี AC/test/demo/sign-off

เพิ่มเติม: ร่างปรับปรุง: WorkshopHub แทนฟอร์มแยก ให้ Student สมัคร workshop ฟรีและดู Coordinator จัดการรายชื่อ เพื่อลดข้อมูลซ้ำ/ยืนยันล่าช้า รุ่นแรกไม่รวม payment/certificates เสนอ metric/baseline, ป้องกันสมัครซ้ำ/ข้อมูลรั่ว, ประเมิน person-days ก่อนรับ deadline และรับงานด้วย AC/tests/demo/sign-off **ตรวจรับ**: Student ทดลองสมัคร/ยกเลิก; Coordinator ตรวจรายชื่อ/รับรองผล; metric/risk/effort ที่เสนอเพิ่มยังต้องตกลง

3) RBAC: default deny / ไม่สืบทอดสิทธิ์

Operation	Student	Coordinator
View published workshops	Allow	Allow
Submit registration	Allow	Deny
Create workshop	Deny	Allow
Edit workshop	Deny	Allow
Export participant list	Deny	Allow

Coordinator ไม่ได้สิทธิ์สมัครเองโดยอัตโนมัติ; คนละ 1 role; ไม่ grant = deny; อย่าเพิ่มเงื่อนไข owner ที่โจทย์ไม่ให้ **Backend**: ยืนยันตัวตน → ตรวจ role/operation → ตรวจข้อมูล → ทำงาน; เพิ่ม owner/state เฉพาะที่ policy กำหนด **เพิ่มเติม:** Negative test: Student เรียก POST /workshops ตรง → 403 และไม่เพิ่มข้อมูล; Coordinator ขอสมัครต้องถูกปฏิเสธ; endpoint/status นี้เป็นแบบออกแบบเพิ่ม

4) Upload attack: ข้าม trust boundary

Browser check ถูกข้าม → script ปลอม PDF → server เก็บแบบบันได้ → เปิด URL → script อ่านรายงานคนอื่น **Actor**: นักศึกษาข้ามการตรวจใน browser; **Assets**: รายงานผู้อื่น/ระบบ; **Surface**: upload API/file URL; **Boundary**: client → server และไฟล์ข้อมูล → โค้ดที่รัน **CIA**: อ่านรายงานคนอื่นละเมิด C; code execution เสียง I/A ด้วย แยกเหตุที่เกิดแล้วจากผลที่อาจเกิด **Control/test 1**: server ตรวจ PDF type/content/size; ส่ง script, MIME ปลอมหรือไฟล์ใหญ่ → 4xx ไม่เก็บเป็นรายงาน **Control/test 2**: เก็บนอก webroot/รันไม่ได้; download handler ตรวจ owner; เปิด script → ไม่รัน; ขอรายงานคนอื่น → ปฏิเสธ **เพิ่มเติม:** สุ่มชื่อไฟล์ ตรวจ path จำกัดสิทธิ์ สแกนมัลแวร์ ตามความเสี่ยงและ log อย่างปลอดภัย; ตรวจใน browser ช่วย UX ไม่ได้แทน server

ตอบ security พร้อมเหตุผล

ช่องโหว่ → จุดเข้า/asset → boundary → CIA → defense → negative check + expected; ไม่ใช่แค่รายชื่อ attack **อย่าสับสน**: XSS รันใน browser; CSRF ใช้ browser/cookie ส่งคำสั่ง; SSRF ให้ server ดึง URL; SQLi เปลี่ยนความหมาย query; เลือกวิธีป้องกันและทดสอบให้ตรงกับช่องโหว่ **Boundary**: จำกัด 3 → ทดสอบ 2/3/4, ข้ามภาค, ยกเลิกแล้วนับไหม? เงื่อนไขที่โจทย์ไม่บอกต้องถาม ไม่เพิ่มกฎเอง **แยกศัพท์**: threat = เหตุร้ายที่อาจเกิด; vulnerability = จุดอ่อน; attack = การใช้จุดอ่อน; risk ประเมินโอกาส/ผลกระทบ; control = มาตรการลดความเสี่ยง

Part 3 · ตัวอย่างการประยุกต์ UML/UI

ลำดับคิด: actor/goal → workflow/scenario → data/UI → state

ระบบ	ข้อเท็จจริงสำคัญก่อนวาด UML/UI
POS	Cashier: items/discount/total; cancel ก่อนจบ; cash/card ผ่าน PSP; declined → retry/cash/cancel; สำเร็จพิมพ์ receipt/ลด stock; return บันทึกเหตุผล/refund/คืน stock; Manager ดู daily cash/card และดูแลสินค้า/ราคา
Hotel	Guest คืนวันเข้า-ออก/เมือง/ประเภท เลือกห้อง/ประเภท จ่าย card/KBank → เลขยืนยัน/ลดห้องว่าง/email ถึง Guest+Hotel; Maps แสดงที่ตั้ง; All-Inn Admin ดูแล users/hotels/types/report; Hotel Admin ดู supply/price/report ของโรงแรมตน; commission 10%; SMTP/Outlook
Ticket	Customer เห็นเฉพาะบริษัทตน; list ตาม software/company เรียง due; ticket มีภาพ 1+ไฟล์ 1; save สร้าง Requested/action แรก; CCE อนุมัติ/ปิด แต่แก้ ticket ไม่ได้; Programmer เริ่ม/อัปเดต/ปิด; action แก้/ลบไม่ได้; IT Admin ดูแลข้อมูลหลัก ไม่ทำ workflow
Grades	Instructor เลือก semester/course/sections ดึง students กรอก/แก้/print sheet; ครบจึง Ready แล้ว submit → Submitted+transcript+email; SSO, Registration, Student Info, Transcript, Email เป็นระบบนอก

Practice Midterm · Part 2

Part 2 รวม 25 คะแนน: PM ทำ WBS/RACI/Gantt; PO ทำ grooming; SM ทำ sprint planning

1) DormFix: WBS 3 + RACI 2 + Gantt 5

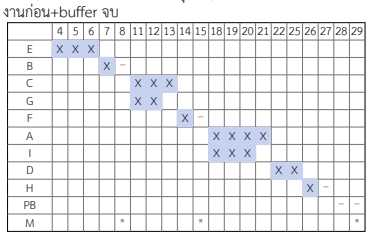
บริบท: DormFix เว็บแจ้งซ่อมหอพัก (Waterfall); PM = Project Manager; Aom = BA, Bank = UX/UI, Chai = Dev, Dao = Dev/Tester; ผจก.หอ (sponsor) อนุมัติ B/ตรวจรับ H; หัวหน้าช่าง (user) อนุมัติ F **ตาราง**: Jan 2027; d = วันทำงาน; Work/Buf = ทำ/เผื่อ

ID	Task · Owner · d	Work / Buf
E	Write requirements (first) · Aom · 3d	4-6 / –
B	Submit requirements for approval · PM · 1d	7 / +8
C	Design screens · Bank · 3d	11-13 / –
G	Design database · Chai · 2d	11-12 / –
F	Submit both designs for approval · PM · 1d	14 / +15
A	Code request form · Chai · 4d	18-21 / –
I	Code technician dashboard · Dao · 3d	18-20 / –
D	Test both coded parts together · Dao · 2d	22,25 / –
H	Submit tested system (acceptance) · PM · 1d	26 / +27
PB	Project buffer · 2d	– / 28-29
C/G: from approved requirements; A/I: from approved designs		

ลำดับ: E → B → (C‖G) → F → (A‖I) → D → H → PB

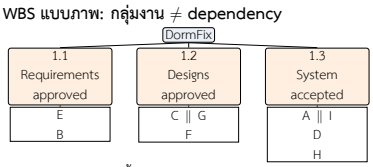
Task	PM	Aom	Bank	Chai	Dao
E	A	R	C	I	–
C	A	C	R	I	–
G	A	–	C	R	I
D	A	I	–	C	R

เพิ่มเติม: ตัวอย่าง RACI; “–” = ไม่มีบทบาทในงานนั้น **กฎ**: เริ่ม จ. 4 Jan; เต็มเวลา จ.-ศ. ไม่มีวันหยุดอื่น; งานถัดไปเริ่มวันทำงานหลังงานก่อน+buffer จบ



M: 8/15/29 Jan; X = งาน, – = buffer, * = milestone (0 วัน); Start/Due = วันแรก/ท้ายรวม buffer เหตุผล: B/F/H รอคนนอกที่มีงานอื่น เผื่อหลังงานละ 1 วัน; ทีมใช้ project buffer ร่วม 2 วันท้าย; milestone ไม่เพิ่มวันทำงาน

WBS แบบภาพ: กลุ่มงาน ≠ dependency



ตรวจ WBS: ใช้ A-I ครั้งเดียว; 1.4 ว่าง; เลขใบ 1.1.1, 1.1.2 ฯลฯ ตามกลุ่ม; ต้นไม้ไม่ใช่ dependency

2) Backlog grooming (10): Issue #12

ในฐานะนักศึกษา ฉันต้องการจองสนามแบดหนึ่งชั่วโมง เพื่อไม่ต้องต่อคิวที่เคาน์เตอร์ (ใช้โครง As a / I need / So that) **รายละเอียด**: มีสนาม/เวลาเปิดแล้ว; นักศึกษาถือการจองได้คนละหนึ่งรายการต่อวัน; Court 2 วันนี้ 17:00 ว่าง

Given Court 2 วันนี้ 17:00 ว่าง
When นักศึกษาจอง Court 2 เวลา 17:00
Then ช่วงนั้นแสดง “Booked” **เพิ่มเติม:** AC ลบ: มี booking วันนี้อยู่แล้ว → ขอเพิ่ม → ปฏิเสธและไม่สร้างรายการ; ถามเงื่อนไขยกเลิกก่อนขยายกฎ **Metadata**: Assignees –; Labels enhancement; Milestone –; Estimate –; ยังไม่ planning อย่ากำหนดคน/points/sprint เอง **Bug variant (Issue #15)**: DB บันทึกการยกเลิกแล้วแต่สนามยัง Booked คนอื่นจองไม่ได้ → label bug; Given Court 1 เวลา 18:00 ถูกจอง; When ยกเลิก; Then “Free”; อีก 3 metadata ใช้ “–” **ถามเพิ่ม**: เขตเวลา? ยกเลิกแล้วคืนสิทธิ์วันนั้นไหม? ไม่เติมเงื่อนไขที่ยังไม่ตกลงเป็น AC บังคับ

3) Sprint planning (5): Goal + capacity

ShuttleNow Sprint 6: goal = นักศึกษาติดตามรถ/รู้เวลาถึง; anchor “ดูรายการเส้นทางรถรับส่ง” = 3 points; Sprint ก่อนเสร็จ 16 คง velocity เดิม; backlog เรียง priority

#	Story / เทียบ anchor	Pt	Pick
1	Live position (+2 ขึ้น)	8	Yes
2	QR fare (เท่ากัน)	3	No
3	Arrival at my stop (+1)	5	Yes
4	Arrival alert (เท่ากัน)	3	Yes
5	Arrival at all stops (+1)	5	No
6	Fix mobile truncation (−1)	2	No

เลือก 1/3/4: 8+5+3=16; 2 ไม่ตรง goal; เพิ่ม 5 รวม 21 เกิน capacity; 6 ไม่มีที่เหลือ. Fibonacci นับขั้น ไม่บวกเลขตรง

PERT: คำนวณ effort ไม่ใช่วันบัดส่ง

ข้อมูล O/M/P (h): T1 guard 2/3/6; T2 queue 3/5/10; T3 workflow 4/6/11; T4 comments 4/7/13; T5 admin 3/5/12; T6 auth/tests 4/6/14 T1: E=3.33 h, SD=0.67 h; รวม E=35.67 h, SD=3.23 h เมื่อประมาณการอิสระ → 35.7 ± 3.2 h (หนึ่ง SD) ไม่ใช่ช่วงรับประกัน; วันจบขึ้นกับงานขนาน/คน

Part 3 · เตรียมครบ 5 แบบ เลือกสอบ 3

ชนิด	เริ่มจากอะไร / ต้องแสดงให้ครบ
1 Sequence	scenario → participants/messages ตามเวลา รวม error/ผลข้างเคียงที่โจทย์ระบุ
2 Activity	workflow → lanes/actions/guards/loops/payment branches/start/end
3 UI	requirements → List+Form, fields, */edit/RO/LoV, controls ตาม role/state
4 Use Case	boundary+actors → goals; include/extend/generalization ให้ที่ศถูก
5 State	object เดียว → states/events/[guards]/effects และข้อห้ามหลังปิด

Practice Midterm · Part 3

UML/UI 30 คะแนน: เลือก 3 จาก 5 แบบ แบบละ 10 คะแนน; เตรียมโครงให้ครบทุกแบบ

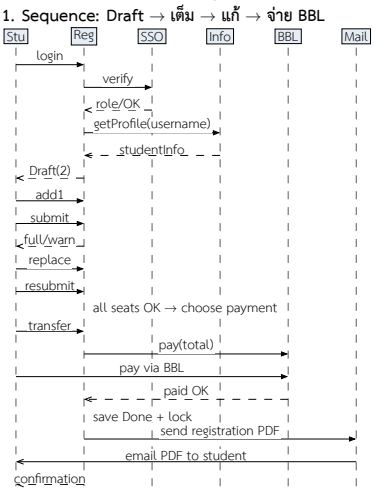
KU Registration: อ่าน R1-R11 ก่อนวาด

R1: KUSSO login/คืน role; **R5**: ฟอร์มภาค active ดึง ID/name/degree/email/username จาก Student Info; **R6-7**: เพิ่ม/กรอง offering แล้วยอดเปลี่ยน; **R8**: Draft บันทึก/แก้ได้; **R9**: ตรวจทุก section เต็ม → แก้/ส่งใหม่; **R10**: card/KBank หรือ transfer/BBL; **R11**: Done/ล็อก/email PDF

Actors (R1-R4)	ขอบเขตที่ระบุในโจทย์
Department Admin	R2 สร้าง course: code/name/description/credits/fee; R3 ตั้ง offerings: semester/code, section/room/time/notes/max students
University Admin	R4 สร้าง semester และกำหนด active semester

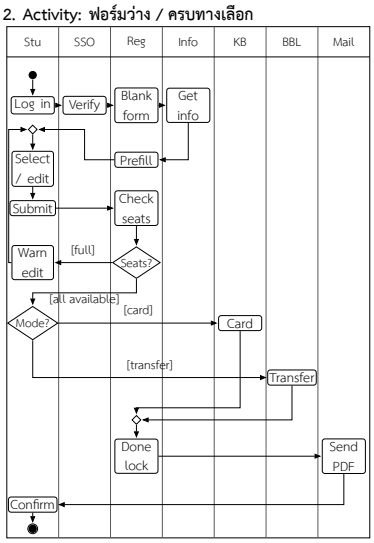
SSO คืน Instructor ได้ แต่ R1-R11 ไม่กำหนดงานให้; ไม่แต่งสิทธิ์ **เพิ่มเติม:** ชื่อข้อความในภาพอธิบายพฤติกรรม ไม่ใช่ชื่อ API

1. Sequence: Draft → เต็ม → แก้ → จ่าย BBL



เริ่มจาก Draft เติม 2 offerings แล้วเพิ่ม 1; ส่งแล้วเต็มก่อนแก้/ส่งซ้ำสำเร็จ; Reg จัดการ DB ภายใน; หากโจทย์ให้ DB lifeline ต้องแสดง call

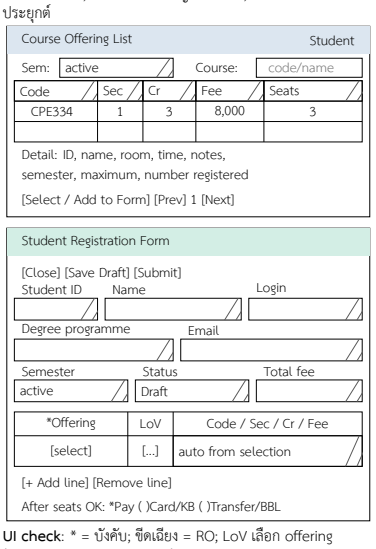
2. Activity: ฟอร์มว่าง / ครบทางเลือก



Stu = Student; Reg = KU Registration; SSO = KUSSO; Info = KU Student Info; Mail = KUEmail; KB = KBank; BBL = Bangkok Bank; จ่ายสำเร็จตามโจทย์ วนเฉพาะแก้ section เต็ม

3. UI: List + Form จากข้อกำหนด

คำสั่ง: Complete เติมข้อมูลในโครงที่กำหนด; Design ออกแบบจากข้อกำหนด; Invoice ใช้ฝึกสัญลักษณ์ UI; KU เป็นตัวอย่างประยุกต์

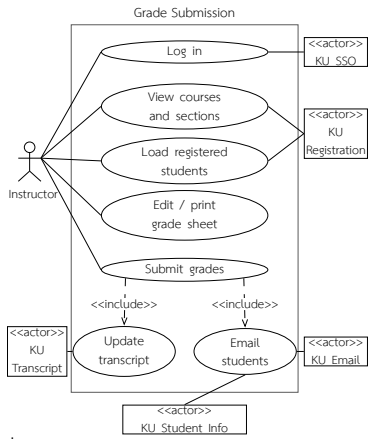


UI check: * = บังคับ; ขีดเยื้อง = RO; LoV เลือก offering (course+semester+section); profile/ภาค/total = RO; R7 ครบใน columns/detail

Practice Midterm · Part 3 (ต่อ)

4. Use Case: Grades

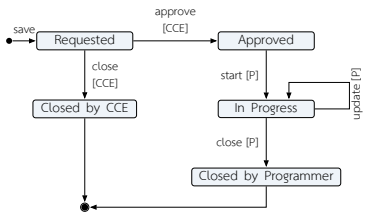
ทีศ: actor = เส้นทึบไม่มีหัว; include base→required; extend extension→base [condition]; generalization child→parent (สามเหลี่ยมโปร่งชี้ parent). แสดง actor/goal ไม่ใช่ลำดับขั้นแบบ flowchart; เลือกความสัมพันธ์ตามข้อกำหนด



เพิ่มเติม: Submit ต้อง include การอัปเดต transcript และส่ง email; โจทย์ Grades ไม่มี actor/use case แบบแม่-ลูก จึงไม่ใช้ generalization

5. State: object เดียว / event [guard] / effect

วิธีทำ: object → states/events/[guards]/effects; initial/final/self-loop; actor ไม่ใช่ state; P = Programmer



Ticket: save [System] → Requested; action อัปเดต Last Status/Due, แก้/ลบไม่ได้; closed ห้ามเพิ่ม; notes/due update เป็น self-loop **เทียบ**: Registration Draft → Done เมื่อจ่าย / lock+email; เต็มยัง Draft. Grades Draft → Ready → Submitted; ห้ามแต่ง state

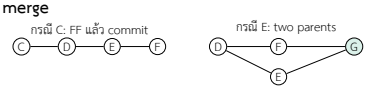
Git: เลือก 2 จาก 3 รูปแบบ ข้อละ 10 คะแนน

รูปแบบ: ทำนาย graph / อธิบาย graph / เขียนคำสั่งแล้ววาด graph ผลลัพธ์ **โจทย์**: clean/คำสั่งสำเร็จ; conflict เฉพาะที่ระบุ. HEAD → branch → commit; origin/* = remote ที่รู้ล่าสุด; tag คงที่

กรณี ฐาน → ผล / HEAD / ตัวชี้ที่ต้องรักษา
A	FF C→E; s=f=E, os=C; switch ท้าย → HEAD/main=B
B	HEAD=s=G, os=F; G ยังไม่ push; f=E แยกสาย → 3-way; graph อย่างเดียวสรุปไม่ได้ว่าจะ conflict
C	s:C→E (FF)→F; HEAD=s=F, f=E, os=C
D	s:D→E (FF)→F; HEAD=s=F, f=E, os=D
E	merge F/E→G (parents F/E); s=G, f=E, os=F; switch ท้าย → HEAD/main=B
F	main=B, origin/main=v1.0=H; switch ยัง B; FF→H; refs อื่นเดิม
G	HEAD=s=H (E/G), os=E; push f ที่ G ไม่ส่ง H; push s → os=H
H	s:C→E; สร้าง/switch f ใหม่→F; s=feature/pass_rate=E, os=C
I	s:D→G (FF)→H (G/F); push→os=H; HEAD=s=H, f=F

ชื่อย่อ: s=staging, f=feature, os=origin/staging; main/origin/main=B ยกเว้น fast-forward ในกรณี F; ใช้ชื่อจริงตามโจทย์

จำ ancestry: fast-forward แล้ว commit / three-way merge



คำสั่ง: แทนชื่อใน < > ด้วยค่าจากโจทย์

git status: branch/staged/unstaged/untracked; อ่านอย่างเดียว. git diff: tracked worktree เทียบ index (ไม่รวม untracked). git diff –cached: index เทียบ HEAD
git add <files>: stage เฉพาะชื่อไฟล์; ยังไม่ commit. git commit -m "<msg>": บันทึก index เป็นลูก HEAD และเลื่อน current branch; ไม่ push
git branch <b>: สร้าง branch ที่ HEAD; HEAD/ไฟล์เดิม. git switch <b>: ย้าย HEAD/ไฟล์เมื่อ clean; ไม่เลื่อน branch pointer
git merge <ref>: รวมเข้า current; FF เลื่อน ref, แยกสายสร้าง 2 parents; source คงเดิม. git merge –ff-only origin/<b>: ยอมเฉพาะ FF
git fetch origin: รับ commits/เลื่อน origin/* เท่านั้น. git push origin <b>: ส่ง branch/อัปเดต origin/b; local HEAD/ไฟล์เดิม **กรณี C**: staging = sprint_1_staging, feature/load_scores; แก้ score_data.py, scores.csv; สร้าง validation_notes.txt; message "Document score validation cases" **กรณี H**: s=sprint_2_staging; merge feature/pass_rate; branch+switch feature/grade_distribution; แก้ report.py, scores.csv; เพิ่ม grade_ranges.csv; message "Add grade distribution report" **กรณี I**: b=sprint_2_staging, ref=feature/export_results; switch b → FF origin/b → merge ref → push b ห้าม git add .; วาด parent/local/origin/tag/HEAD ทุกขั้น

วิศิษฐ์ สุวรรณเนาว์ 67070501042